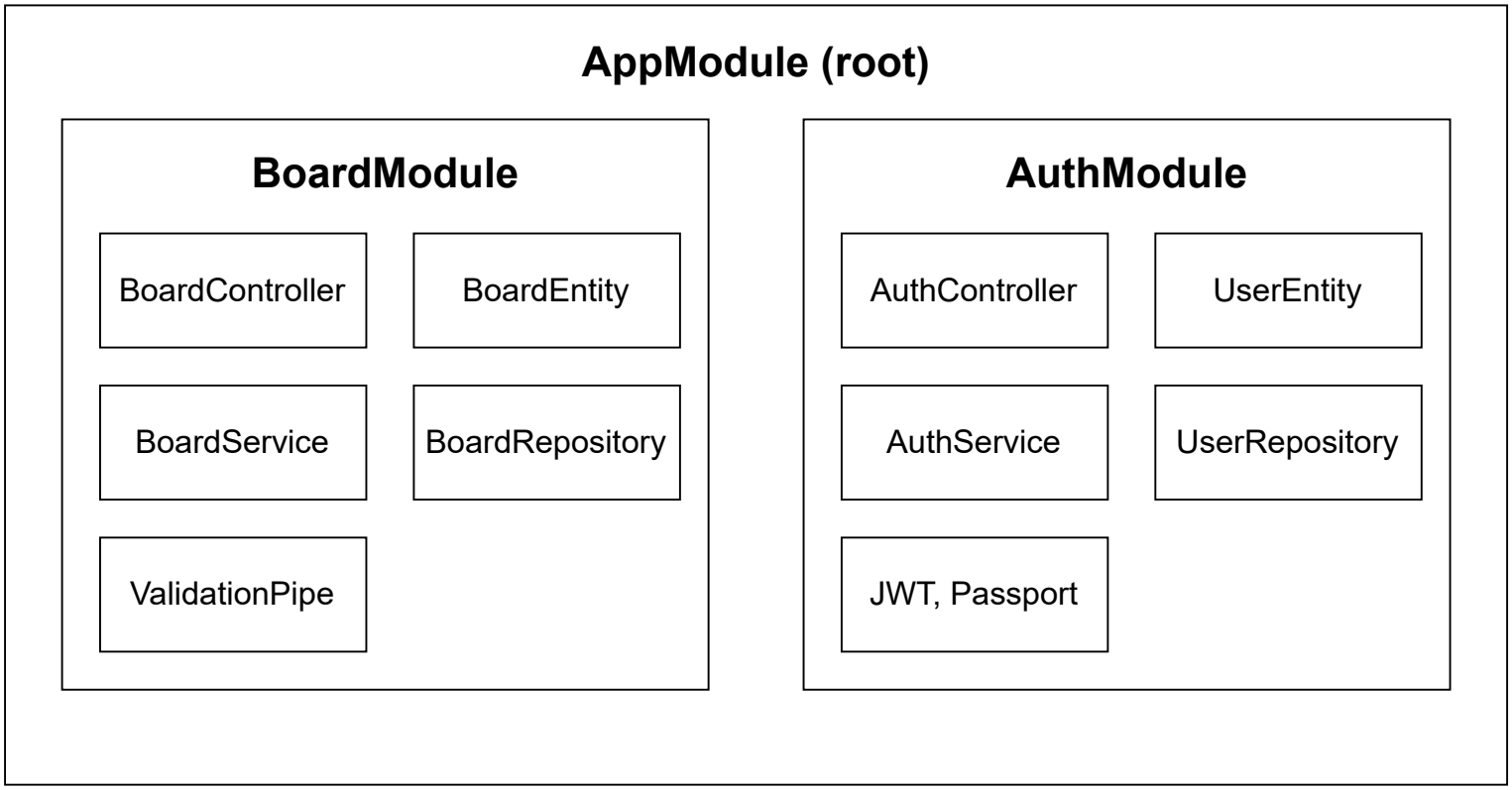
인증 기능 구현을 위한 준비

이번 시간 부터는 인증기능을 구현하겠습니다.



CLI를 이용한 모듈, 컨트롤러, 서비스 생성

- nest g module auth
auth 모듈 생성

- nest g controller auth --no-spec
auth 컨트럴로 생성

- nest g service auth --no-spec
-auth 서비스 생성

User를 위한 Entity 생성

유저에 대한 인증을 하는 것이니 유저가 필요합니다.
그래서 유저 데이터를 위한 유저 Entity를 생성해주겠습니다.

1. user.entity.ts 파일 생성

2. 파일 소스 코드 작성

user.entity.ts

```
import { BaseEntity, Column, Entity, Pri

@Entity()
export class User extends BaseEntity {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    username: string;

    @Column()
    password: string;
}
```

Repository 생성

User Entity를 생성, 수정, 삭제등의 로직을 처리하기 위해서 Repository를 생성합니다.

1. user.repository.ts 파일 생성

2. 소스 코드 작성

user.repository.ts

```
import { EntityRepository, Repository } from "typeorm";
import { User } from "./user.entity";

@EntityRepository(User)
export class UserRepository extends Repository<User> {

}
```

생성된 User Repository를 다른곳에서 사용하기 위해..

- auth module에서 imports 안에 UserRepository를 넣어줍니다.

auth.module.ts

```
@Module({
  imports: [
    TypeOrmModule.forFeature([UserRepository])
  ],
  controllers: [AuthController],
  providers: [AuthService]
})
export class AuthModule {}
```

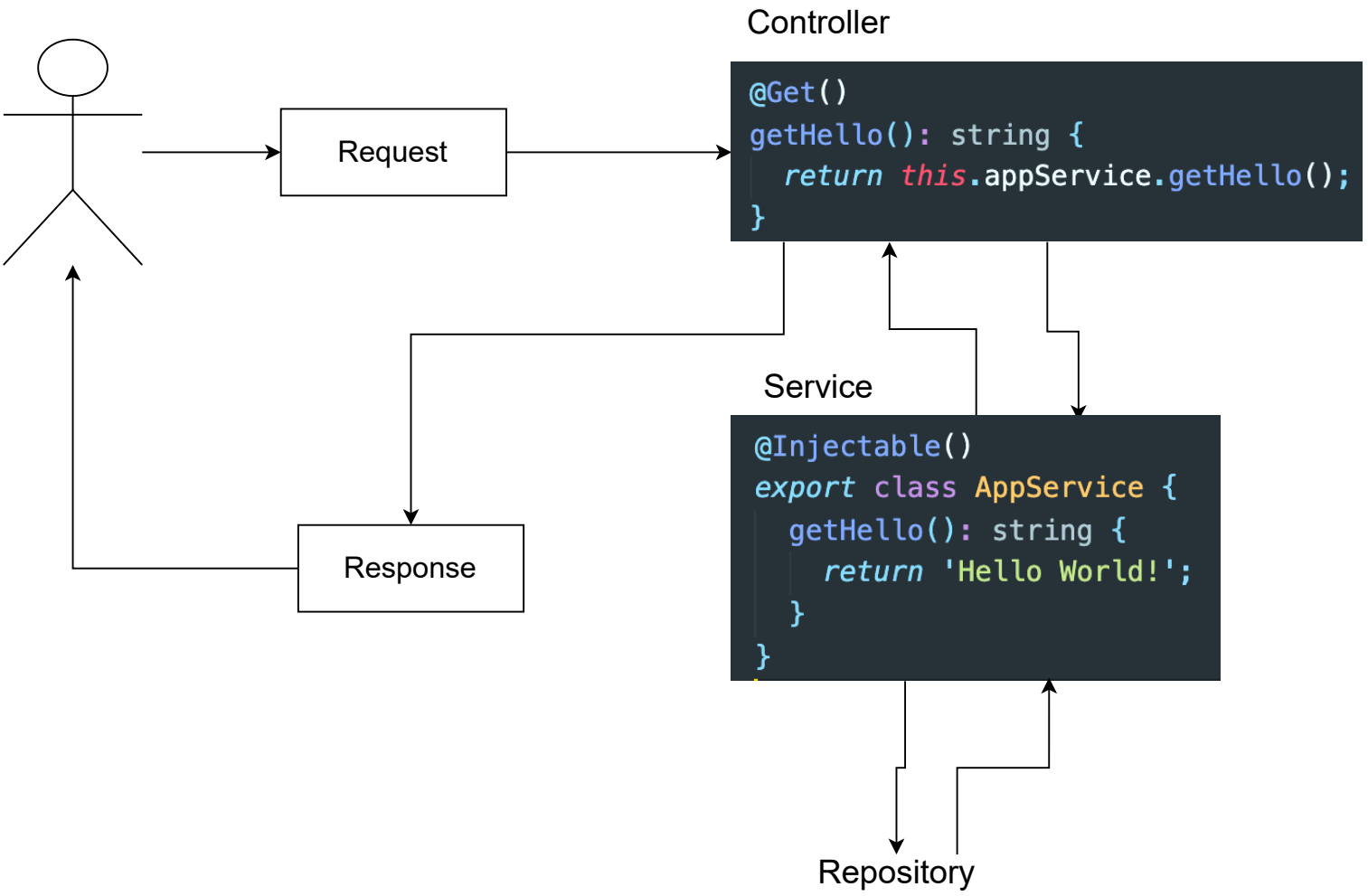
Repository Injection

User Repository를 auth Sercive 안에서 사용하기 위해서 유저 리포지터리를 넣어주겠습니다.

```
@Injectable()
export class AuthService {
    constructor(
        @InjectRepository(UserRepository)
        private userRepository: UserRepository,
    ) {}
}
```

회원 가입 기능 구현

이번 시간에는 유저 회원 가입 기능을 구현하겠습니다.



user.repository.ts

```
@EntityRepository(User)
export class UserRepository extends Repository<User> {
    async createUser(authCredentialsDto: AuthCredentialsDto): Promise<voic
        const {username, password} = authCredentialsDto;
        const this.create({ username, password});
        await this.save(user);
    }
}
```

auth/dto/auth-credential.dto.ts

```ts
export class AuthCredentialsDto {
    username: string;
    password: string;
}
```

user.service.ts

```ts
async signUp(authCredentialsDto: AuthCredentialsDto): Promise<void> {
    return this.userRepository.createUser(authCredentialsDto);
}
```

user.controller.ts

```ts
@Controller('auth')
export class AuthController {
    constructor(
        private authService: AuthService,
    ){}

    @Post('/signup')
    signUp(@Body() authCredentialsDto: AuthCredentialsDto ): Promise<void> {
        return this.authService.signUp(authCredentialsDto);
    }
}
```

유저 데이터 유효성 체크

이번 시간에는 유저를 생성할 때 원하는 이름의 길이 비밀번호 길이 같은 유효성 체크를 할 수 있게 각 Column에 맞는 조건을

Class-validator

https://github.com/typestack/class-validator

- 유효성 체크를 하기 위해서는 class-validator 모듈을 이용하시면 됩니다.

- Dto 파일에서 Request로 들어오는 값을 정의해주고 있기 때문에 Dto 파일에 값들 하나하나에 class-validator를 이용해서 유효성 조건을 넣어주겠습니다.

auth-credentials.ts

```ts
import { IsString, Matches, MaxLength,

export class AuthCredentialsDto {
    @IsString()
    @MinLength(4)
    @MaxLength(20)
    username: string;

    @IsString()
    @MinLength(4)
    @MaxLength(20)
    // 영어랑 숫자만 가능한 유효성 체크
    @Matches(/^[a-zA-Z0-9]*$/)
    password: string;
}
```

ValidationPipe

요청이 컨트롤러에 있는 핸들러로 들어왔을 때 Dto에 있는 유효성 조건에 맞게 체크를 해주려면 ValidationPipe을 넣어주셔야 합니다.

auth.controller.ts

```ts
@Post('/signup')
```

```
@Post('/signup')
signUp(@Body(ValidationPipe) authCredentialsDto: AuthCredentialsDto):
    return this.authService.signUp(authCredentialsDto);
}
```

POST localhost:3000/auth/signup

Params Auth Headers (9) Body Pre-req. Tests Settings

raw JSON

```
1  {
2      "username": "john",
3      "password": "하하"
4  }
```

Body 400 Bad Request 7 ms 531 B S

Pretty Raw Preview Visualize JSON

```
1  {
2      "statusCode": 400,
3      "error": "Bad Request",
4      "message": [
5          {
6              "target": {
7                  "username": "john",
8                  "password": "하하"
9              },
10             "value": "하하",
11             "property": "password",
12             "children": [],
13             "constraints": {
14                 "matches": "password must match /^[a-zA-Z0-9]*$/ regular
15                 "minLength": "password must be longer than or equal to 4
16             }
```

POST localhost:3000/auth/signup

Params Auth Headers (9) Body Pre-req. Te

raw JSON

```
1  {
2      "username": "john",
3      "password": "1234567"
4  }
```

Body

Pretty Raw Preview Visualize Text

```
1
```

	id [PK] integer	username character varying	password character varying
1	1	john	1234567

원하는 에러 메시지 설정하기

현재 비밀번호가 유효성체크에 걸리면 엄청 긴 에러메시지가 나옵니다.
이 부분을 원하는 메시지로 사용자에게 설정해주겠습니다.

user.entity.ts

```
  @IsString()
  @MinLength(4)
  @MaxLength(20)
  // 영어랑 숫자만 가능한 유효성 체크
  @Matches(
      /^[a-zA-Z0-9]*$/,
      { message: 'Only alphabets and numbers allowed' }
  )
  password: string;
```

유저 이름에 유니크한 값 주기

이번 시간에는 유저를 생성할 때 유저 이름이 이미 사용 되는 유저 이름을 사용하려 한다면 에러를 보내는 기능을 구현해보겠습

두가지 방법

이 기능을 구현하기 위해서는 두가지 방법이 있습니다.

1. repository 에서 findOne 메소드를 이용해서 이미 같은 유저 이름을 가진 아이디가 있는지 확인 하고 없다면 데이터를 저장하는 방법입니다. 하지만 이 방법은 데이터베이스 처리를 두번 해줘야 합니다.

2. 데이터베이스 레벨에서 만약 같은 이름을 가진 유저가 있다면 에러를 던져주는 방법입니다

두번째 방법으로 구현

- user.entity.ts에서 원하는 유니크한 값을 원하는 필드 값을 정해주시면 됩니다.

```
@Entity()
@Unique(['username'])
export class User extends BaseEntity {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    username: string;

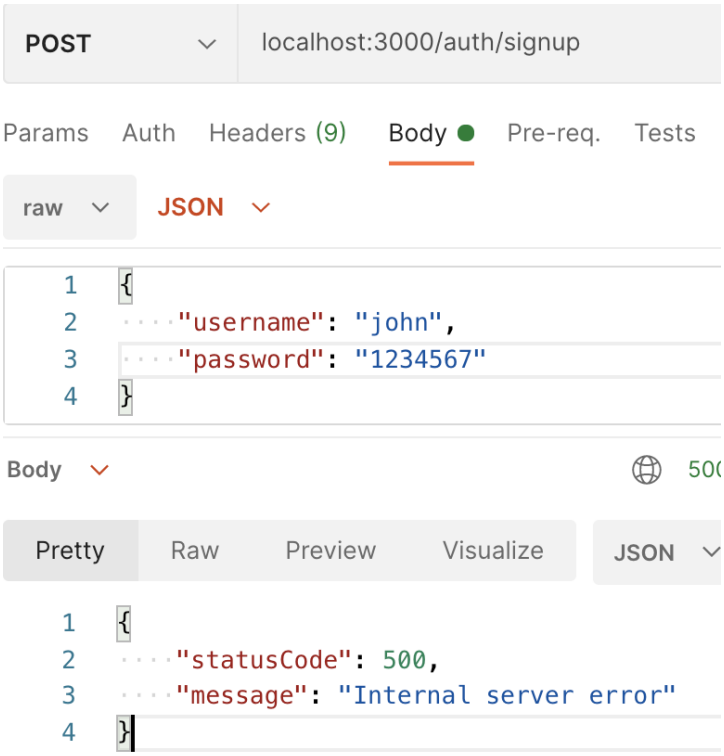
    @Column()
    password: string;
}
```

테스트

- 이미 있는 유저를 다시 생성하려 하면 아래와 같이 에러가 나옵니다. 하지만 그냥 500 에러를 던져버립니다.

- 그 이유는 Nest JS에서 에러가 발생하고 그걸 try catch 구문인 catch에서 잡아주지 않는 다면 이 에러가 Controller 레벨로 가서 그냥 500 에러를 던져 버립니다.

- 이러한 이유 때문에 try catch 구문으로 에러를 잡아줘야합니다.



Try Catch

- 이미 있는 유저를 다시 생성하려 하면 아래와 같이 에러가 나옵니다. 하지만 그냥 500 에러를 던져버립니다.

- 그 이유는 Nest JS에서 에러가 발생하고 그걸 try catch 구문인 catch에서 잡아주지 않는 다면 이 에러가 Controller 레벨로 가서 그냥 500 에러를 던져 버립니다.

- 이러한 이유 때문에 try catch 구문으로 에러를 잡아줘야합니다.

user.repository.ts

```
@EntityRepository(User)
export class UserRepository extends Repository<User> {
    async signUp(authCredentialsDto: AuthCredentialsDto) : Promise<void> {
        const { username, password} = authCredentialsDto;

        const user = new User();
        user.username = username;
        user.password = password;

        try {
            await user.save();
        } catch (error) {
```

```
            if(error.code === '23505') {
                throw new ConflictException('Existing username');
            } else {
                throw new InternalServerErrorException();
            }
        }

    }
}
```

비밀번호 암호화 하기

이번 시간에는 유저를 생성할 때 현재는 비밀번호가 그대로 데이터베이스에 저장됩니다. 그래서 비밀번호를 암호화 해서 저장을 하는 부분을 구현해주겠습니다.

bcryptjs

이 기능을 구현하기 위해서 bcryptjs 라는 모듈을 사용하겠습니다.

npm install bcryptjs --save

import * as bcrypt from 'bcryptjs';

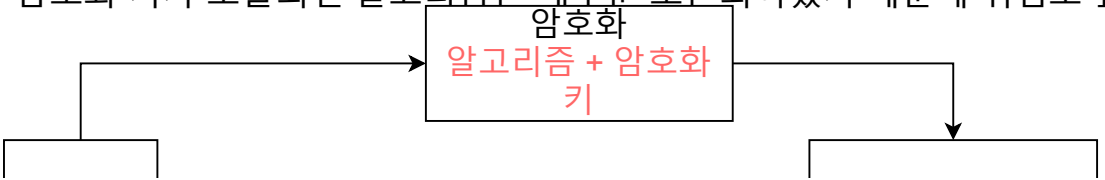
비밀번호를 데이터베이스에 저장하는 방법

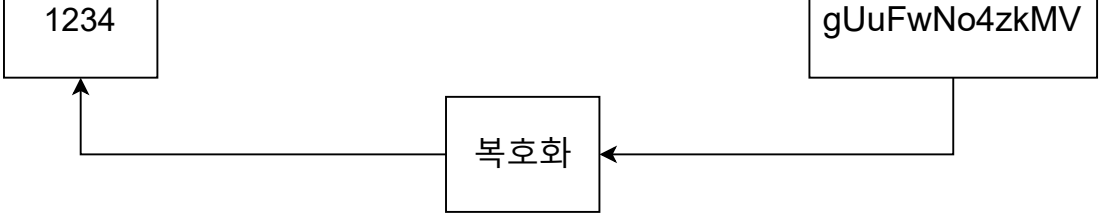
1. 원본 비밀번호를 저장 (최악)

- 1234 ====> 1234

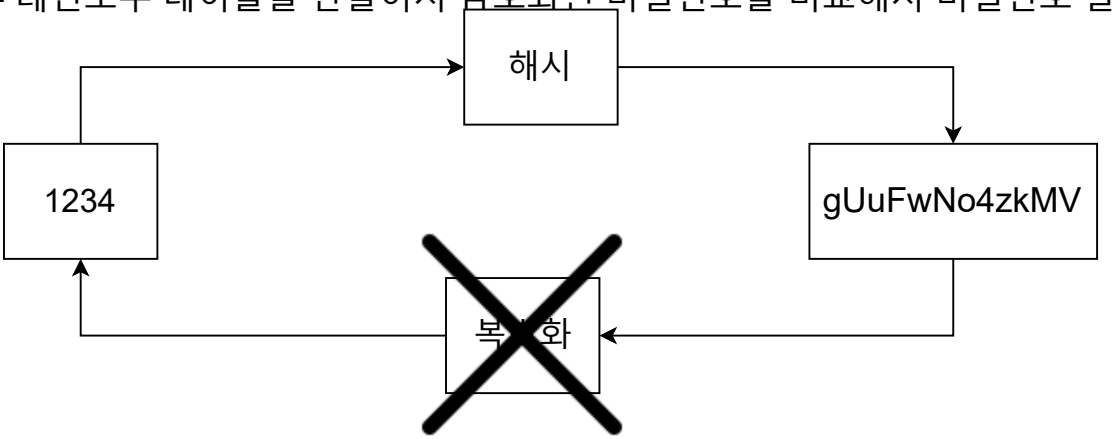
2. 비밀번호를 암호화 키(Encryption Key)와 함께 암호화 (양방향)

- 어떠한 암호를 이용해서 비밀번호를 암호화 하고 그 암호를 이용하여 복호화도 가능
-https://www.online-toolz.com/tools/text-encryption-decryption.php
- 1234 ====>gUuFwNo4zkMV+erdGtBlf5NunNgcELQuiCFJmCU4F+E=
- gUuFwNo4zkMV+erdGtBlf5NunNgcELQuiCFJmCU4F+E= ====> 1234
- 암호화 키가 노출되면 알고리즘은 대부분 오픈되어있기 때문에 위험도 높음





3. SHA256등으로 해시(Hash)해서 저장 (단방향)
- 1234 ====> 03ac674216f3e15c761ee1a5e255f067953623c8b3...
- https://emn178.github.io/online-tools/sha256.html
- 레인보우 테이블을 만들어서 암호화된 비밀번호를 비교해서 비밀번호 알아냄



레인보우 테이블

1234 ====> 03ac674216f3e15c761ee1a5e255f067953623c8b3
letmein ==> 1c8bfe8f801d79745c4631d09fff36c82aa37fc4cce4f
등등등 ... 대부분 유저들은 비슷한 암호를 사용
A 유저 비밀번호 1234 === 03ac674216f3e15c761ee1a5e255f067953623c8b3
B 유저 비밀번호 1234 === 03ac674216f3e15c761ee1a5e255f067953623c8b3

5. 솔트(salt) + 비밀번호(Plain Password)를 해시Hash로 암호화 해서 저장
암호화할 때 원래 비밀번호에다 salt를 붙인 후에 해시로 암호화를 한다.
예) 1234 =====> salt_1234
letmein ====> salt_letmein

A 유저 비밀번호 1234 ===> kenfuduWssW_1234 ======>

user.repository.ts

```
export class UserRepository extends Repository<User> {
    async signUp(authCredentialsDto: AuthCredentialsDto): Promise<void> {
        const { username, password } = authCredentialsDto;

        const salt = await bcrypt.genSalt();
        console.log('salt', salt)
        const hashedPassword = await bcrypt.hash(password, salt)
        console.log('hashedPassword', hashedPassword)
```

```
        const user = new User();
        user.username = username;
        user.password = hashedPassword;

        try {
            await user.save();
        } catch (error) {
            if (error.code === '23505') {
                throw new ConflictException('Existing username');
            } else {
                throw new InternalServerErrorException();
            }
        }

    }
```

비밀번호 암호화 하기(소스코드구현)

bcrypt를 이용해서 비밀번호를 암호화한 후 데이터베이스에 저장하는 부분을 구현하겠습니다.

user.repository.ts

```
async createUser(authCredentialsDto: AuthCredentialsDto): Promise<void> {
    const { username, password } = authCredentialsDto;

    const salt = await bcrypt.genSalt();
    const hashedPassword = await bcrypt.hash(password, salt);
    const user = this.create({ username, password: hashedPassword });

    try {
        await this.save(user);
    } catch (error) {
        if(error.code === '23505') {
            throw new ConflictException('Existing username');
        } else {
            throw new InternalServerErrorException();
        }
    }
```

로그인 기능 구현하기

회원 가입을 하고 비밀번호도 암호화 했기 때문에 가입한 아이디로 로그인을 하는 기능을 구현해보겠습니다.

user.service.ts

```
async signIn(authCredentialsDto: AuthCredentialsDto): Promise<string> {
    const { username, password} = authCredentialsDto;
    const user = await this.userRepository.findOne({ username });

    if(user && (await bcrypt.compare(password, user.password))) {
        return 'logIn success'
    } else {
        throw new UnauthorizedException('logIn failed')
    }
}
```

user.controller.ts

```
@Post('/signin')
signIn(@Body(ValidationPipe) authCredentialsDto: AuthCredentialsDto) {
    return this.authService.signIn(authCredentialsDto)
}
```

JWT에 대하여

이번시간에는 로그인을 할 때 그 로그인한 고유 유저를 위한 토큰을 생성해야 하는데 그 토큰을 생성할 때 JWT라는 모듈을 사용합니다. 이 JWT모듈에 대해서 알아보겠습니다.

JWT란 무엇인가요?

JWT (JSON Web Token)는 당사자간에 정보를 JSON 개체로 안전하게 전송하기위한 컴팩트하고 독립적인 방식을 정의하는 개방형 표준 (RFC 7519)입니다. 이 정보는 디지털 서명이되어 있으므로 확인하고 신뢰할 수 있습니다.

- 간단하게 얘기하자면 정보를 안전하게 전할 때 혹은 유저의 권한 같은 것을 체크를 하기 위해서 사용하는데 유용한 모듈입니다.

JWT의 구조

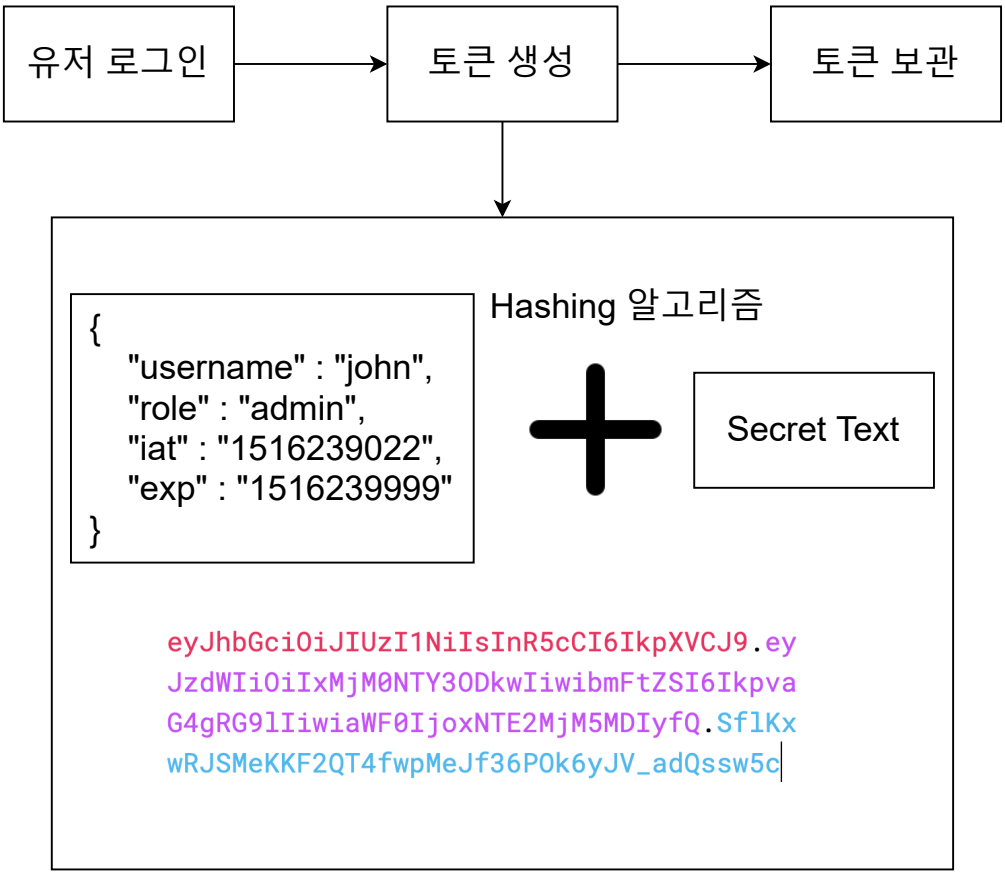
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpva G4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

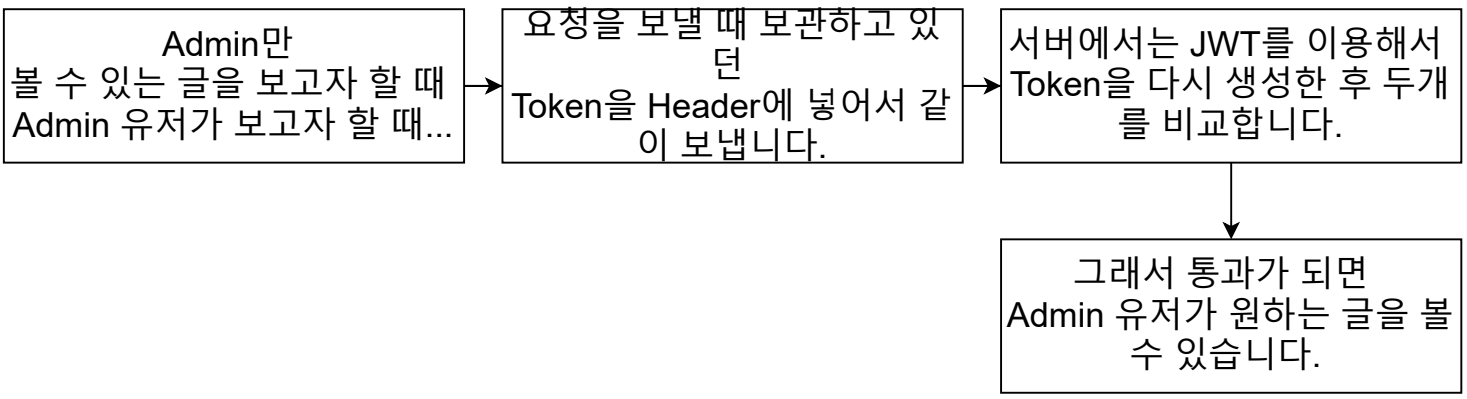
Header
토큰에 대한 메타 데이터를 포함하고 있습니다.
(타입, 해싱 알고리즘 SHA256, RSA ....)

Payload
유저 정보(issuer), 만료 기간(expiration time), 주제(subject) 등등...

Verify Signature
JWT의 마지막 세그먼트는 토큰이 보낸 사람에 의해 서명되었으며 어떤 식으로든 변경되지 않았는지 확인하는 데 사용되는 서명입니다. 서명은 헤더 및 페이로드 세그먼트, 서명 알고리즘, 비밀 또는 공개 키를 사용하여 생성됩니다.

JWT 사용 흐름


유저 로그인
토큰 생성
토큰 보관
{
"username" : "john",
"role" : "admin",
"iat" : "1516239022",
"exp" : "1516239999"
}
Hashing 알고리즘
Secret Text
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpva
G4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKx
wRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c


Admin만
볼 수 있는 글을 보고자 할 때
Admin 유저가 보고자 할 때...
요청을 보낼 때 보관하고 있던
Token을 Header에 넣어서 같이 보냅니다.
서버에서는 JWT를 이용해서
Token을 다시 생성한 후 두개를 비교합니다.
그래서 통과가 되면
Admin 유저가 원하는 글을 볼 수 있습니다.

비교하는 과정

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpvan G4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

secret Text



서버에서 요청에서 같이 온 headers 랑 payload 를 가져오고 서버안에 가지고 있는 Secret을 이용해서 Signature 부분을 다시 생성합니다.
그래서 그 둘이 일치하면 통과가 됩니다.

sdlKxwRJSMeKKF2Q....

sfewfwefRJSMeKKF2Q....

클라이언트에서 온 Headers + 클라이언트에서 온 Payload + 서버에서 가지고 있는 Secret Text

JWT를 이용해서 토큰 생성하기

이번시간에는 저번 시간에 알아본 JWT 모듈을 이용해서 토큰을 생성해보겠습니다. 하지만 Passport라는 모듈도 함께 사용할 것인데 그 이유는 이 모듈이 저번 시간에 JWT를 이용해서 인증 처리하고 하는 등의 과정을 훨씬 쉽게 만들어 주기 때문입니다.

필요한 모듈들 설치하기

@nestjs/jwt
- nestjs에서 jwt를 사용하기 위해 필요한 모듈

@nestjs/passport
- nestjs에서 passport를 사용하기 위해 필요한 모듈

passport
- passport 모듈

passport-jwt
- jwt 모듈

npm install @nestjs/jwt @nestjs/passport passport passport-jwt --save

애플리케이션에 JWT 모듈 등록하기

1. auth 모듈 imports에 넣어주기

```
@Module({
  imports: [
    JwtModule.register({
      secret:'Secret1234',
      signOptions:{
        expiresIn: 60 * 60,
      }
    }),
    TypeOrmModule.forFeature([UserRepository])
  ],
```

```
  controllers: [AuthController],
  providers: [AuthService]
})
export class AuthModule {}
```

Secret
- 토큰을 만들 때 이용하는 Secret 텍스트(아무 텍스트나 넣어줘도 됩니다)

ExpiresIn
- 정해진 시간 이후에는 토큰이 유효하지 않게 됩니다. 60 * 60 은 한시간 이후에는 이 토큰이 더 이상 유효하지 않게 됩니다

애플리케이션에 Passport 모듈 등록하기

1. auth 모듈 imports에 넣어주기

```
@Module({
  imports: [
    PassportModule.register({ defaultStrategy: 'jwt' }),
    JwtModule.register({
      secret: 'Secret1234',
      signOptions: {
        expiresIn: 60 * 60,
      }
    }),
    TypeOrmModule.forFeature([UserRepository])
  ],
  controllers: [AuthController],
  providers: [AuthService]
})
```

로그인 성공 시 JWT를 이용해서 토큰 생성해주기 !!!

1. Service 에서 SignIn 메소드에서 생성해주면 됩니다.
auth 모듈에 JWT를 등록해주었기 때문에 Service에서 JWT를 가져올 수 있습니다.

```
export class AuthService {
    constructor(
        @InjectRepository(UserRepository)
        private userRepository: UserRepository,
        private jwtService: JwtService
    ) { }
```

2. Token을 만드려면 Secret과 Payload가 필요합니다. Payload에는 자신이 전달하고자 하는 정보를 넣어주시면 됩니다. Role 정보든, 유저 이름이든, 이메일이든....하지만 Sensitive한 정보는 넣으시면 안됩니다.
이렇게 Payload를 이용해서 JWT에서 토큰을 만들 때 사용하는 Sign 메소드를 이용해서 토큰을 생성해줍니다.

```
async signIn(authCredentialsDto: AuthCredentialsDto): Promise<{accessToken: string}>  {
    const username = await this.userRepository.signIn(authCredentialsDto);
    if (!username) {
        throw new UnauthorizedException('Invalid credentials');
    }

    const payload: JwtPayload = { username };
    const accessToken = await this.jwtService.sign(payload);

    return { accessToken };
}
```

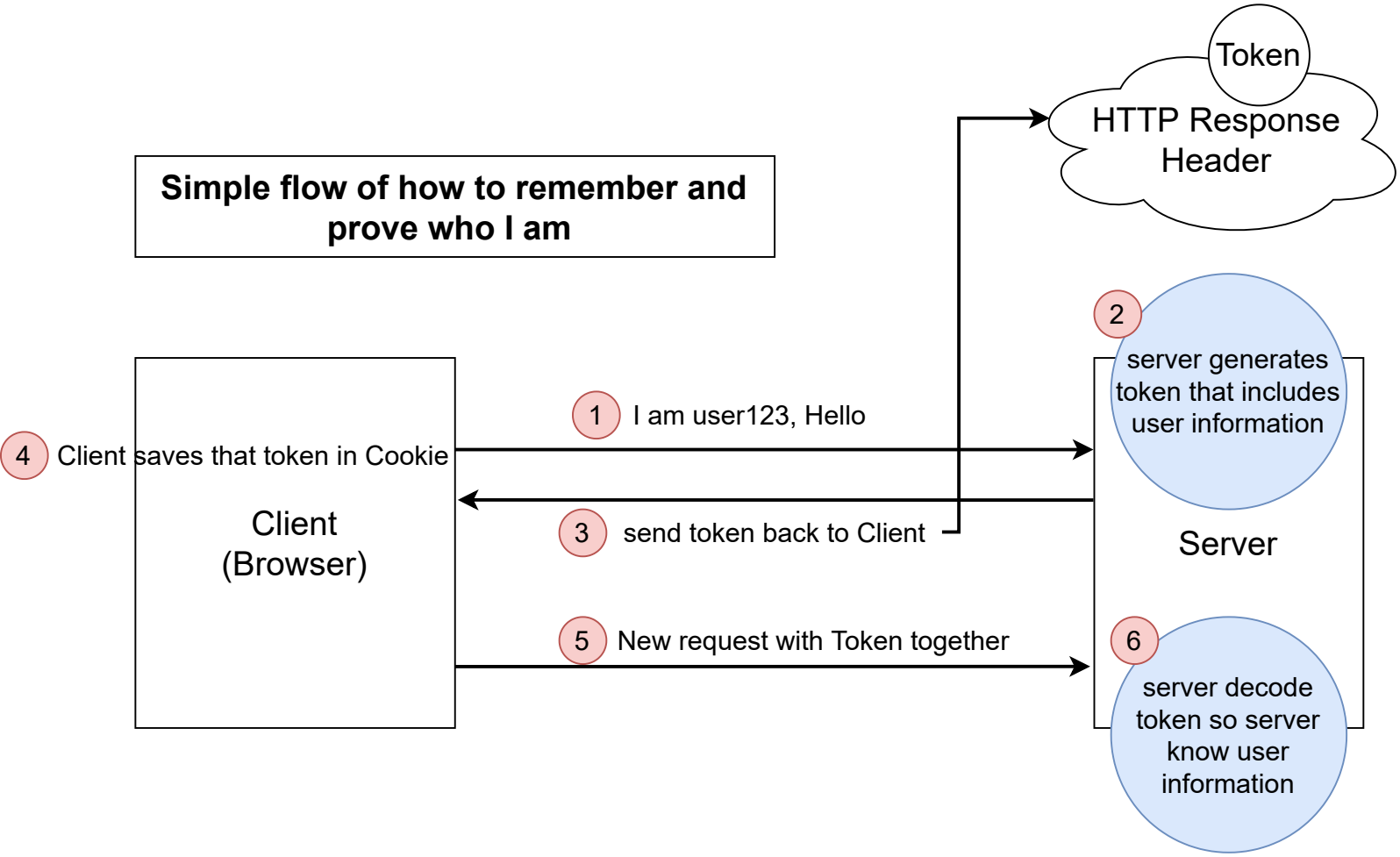
```
export interface JwtPayload {
    username: string;
}
```

jwt-payload.interface.ts

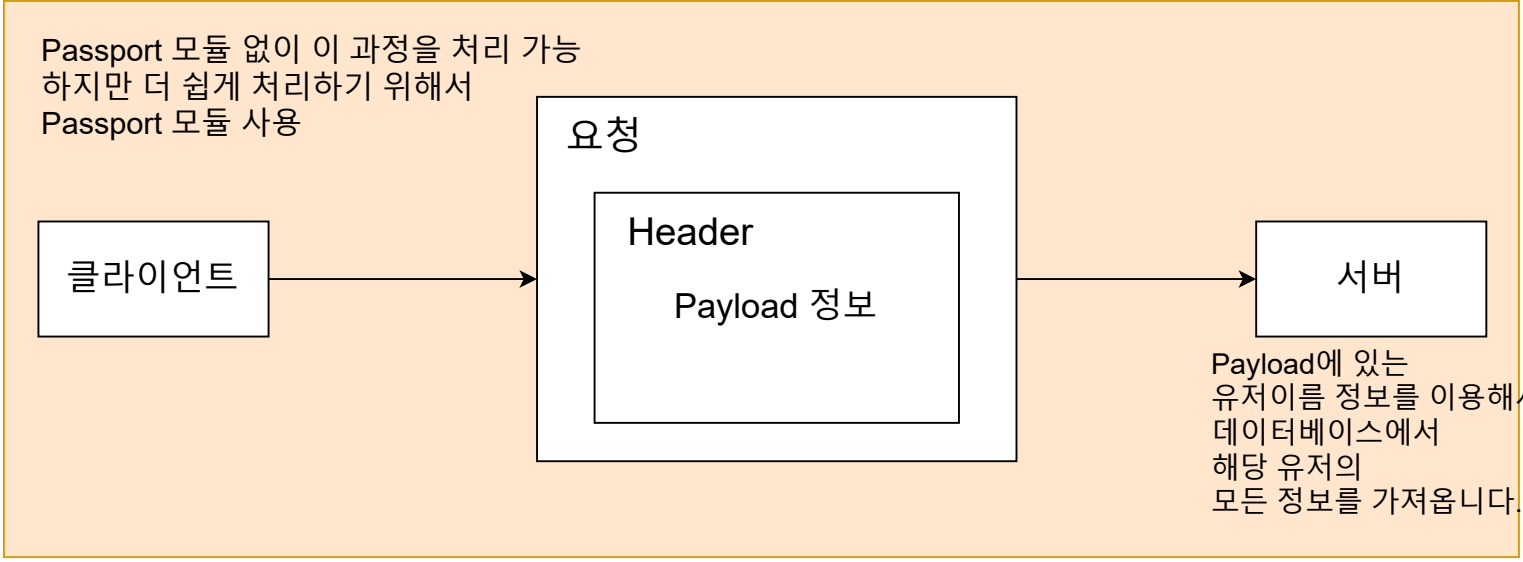
```
@Post('/signin')
signIn(@Body(ValidationPipe) authCredentialsDto: AuthCredentialsDto): Promise<{accessToken: string}> {
    return this.authService.signIn(authCredentialsDto)
}
```

Passport, Jwt 이용해서 토큰 인증 후 유저 정보 가져오기

저번 시간에 JWT를 이용해서 유저가 로그인 할 때 토큰을 생성해줬습니다. 그래서 이제는 그 유저가 요청을 보낼 때 그 요청 안에 있는 Header에 토큰을 넣어서 요청을 보내는데 요청 안에 Payload가 있습니다. 그리고 payload 안에 유저 이름을 넣어줬습니다. 그리고 토큰이 유효한 토큰인지 서버에서 secret text를 이용해서 알아내면 payload 안에 유저 이름을 이용해서 데이터베이스 안에 있는 유저 이름에 해당하는 유저 정보를 모두 가져올수 있습니다. 이러한 처리를 쉽게 해주는게 Passport 모듈 입니다. 그래서 Passport 모듈을 이용해서 이 부분을 구현해보겠습니다



현재 해야하는 부분은 5번과 6번 부분입니다.
토큰을 가지고 요청을 보낼때 서버에서 그 토큰이 유효(valid)한 것인지 체크한 후 만약 유효한것이면 payload 안에 들어있는 username을 이용해서 데이터베이스에 있는 유저인지 체크합니다.
있는 유저라면 유저 객체를 데이터베이스에서 가져옵니다. 없는 유저라면 에러를 보냅니다.



위에 과정을 구현하는 순서

- @types/passport-jwt 모듈
(passport-jwt 모듈을 위한 타입 정의 모듈)

1. jwt.strategy.ts 파일 생성

```ts
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
    constructor(
        @InjectRepository(UserRepository)
        private userRepository: UserRepository
    ) {
        super({
            secretOrKey: 'Secret1234',
            jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken()
        })
    }

    async validate(payload) {
        const { username } = payload;
        const user: User = await this.userRepository.findOne({ username });

        if (!user) {
            throw new UnauthorizedException();
        }

        return user;
    }
}
```

```ts
@Injectable()
// Nest.js can inject it anywhere this service is needed
// via its Dependency Injection system.
export class JwtStrategy extends PassportStrategy(Strategy) {
    // The class extends the PassportStrategy class defined by @nestjs/passport package
    // you're passing the JWT Strategy defined by the passport-jwt Node.js package.
    constructor(
```

```
        @InjectRepository(UserRepository)
        private userRepository: UserRepository
    ) {
        // passes two important options
        super({
            secretOrKey: 'Secret1234',
            // This configures the secret key that JWT Strategy will use
            // to decrypt the JWT token in order to validate it
            // and access its payload.
            jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken()
            // This configures the Strategy (imported from passport-jwt package)
            // to look for the JWT in the Authorization Header of the current Request
            // passed over as a Bearer token.
        })
    }

    //위에서 토큰이 유효한지 체크가 되면 validate 메소드에서 payload에 있는 유저이름이 데이터베이스에서
    //있는 유저인지 확인 후 있다면 유저 객체를 return값으로 던져줍니다.
    // return 값은 @UseGuards(AuthGuard())를 이용한 모든 요청의 Request Object에 들어갑니다.
    async validate(payload) {
        const {username} = payload;
        const user: User = await this.userRepository.findOne({ username });

        if(!user) {
            throw new UnauthorizedException();
        }

        return user;
    }
}
```

방금 만든 JwtStrategy를 사용하기 위해서

AuthModule Providers 항목에 넣어주고 다른 곳에서도 JwtStrategy와 PassportModule 도 사용해줘야 하기 때문에 exports항목에도 넣어줍니다.

```
@Module({
  imports: [
    //유저를 인증하기위해 사용할 기본 strategy를 명시해 주기, 여기서는 jwt
    PassportModule.register({ defaultStrategy: 'jwt'}),
    //jwt 인증 부분을 담당, 그리고 주로 sign()를 위한 부분이다.  secret
    JwtModule.register({
      secret:'Secret1234',
      signOptions:{
        expiresIn: 3600
      }
    }),
    TypeOrmModule.forFeature([UserRepository])
  ],
  controllers: [AuthController],
  //JwtStrategy를 이 Auth 모듈에서 사용할수있게 등록
```

```
  providers: [AuthService, JwtStrategy],
  // JwtStrategy, PassportModule를 다른 모듈에서 사용할수있게 등록
  exports: [JwtStrategy, PassportModule]
})
export class AuthModule {}
```

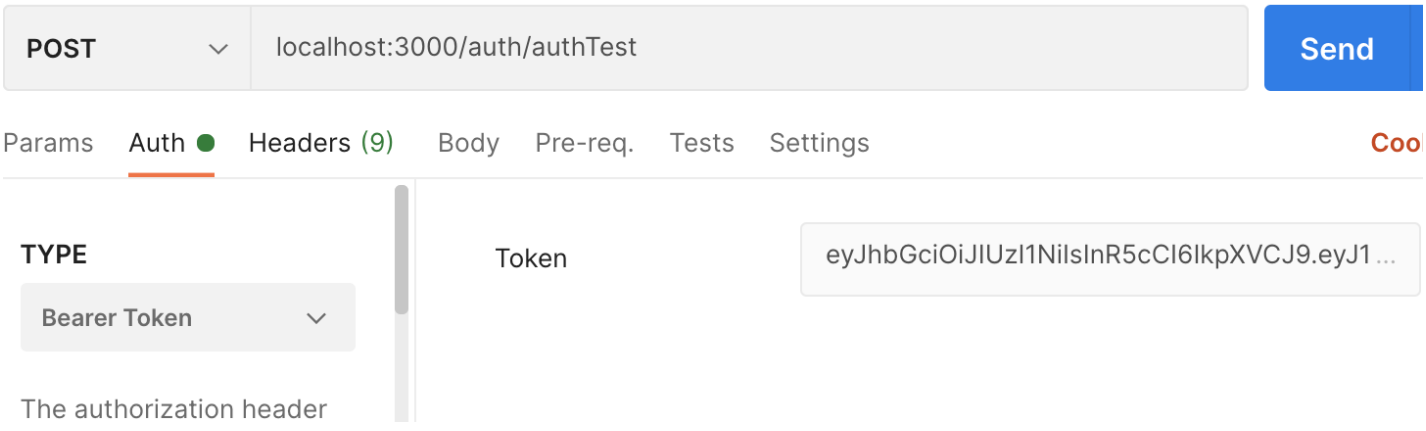
요청안에 유저 정보(유저 객체)가 들어가게 하는 방법

validate 메소드에서 return 값을 user 객체로 주었습니다. 그래서 요청 값안에 user 객체가 들어있으면 하는데 현재 요청을 보낼때는 user 객체가 없습니다. 어떠한 방식으로 가질수있나요 ?!

UseGuards

UseGuards안에 @nestjs/passport에서 가져온 AuthGuard()를 이용하면 요청안에 유저 정보를 넣어줄수있습니다.

```
@Post('/authTest')
@UseGuards(AuthGuard())
authTest(@Req() req) {
    console.log(req);
}
```



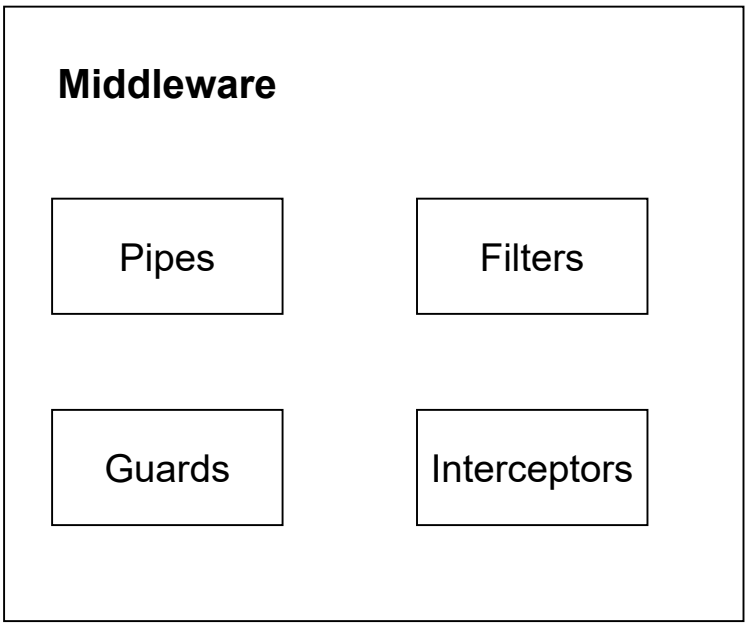

```
route:
```

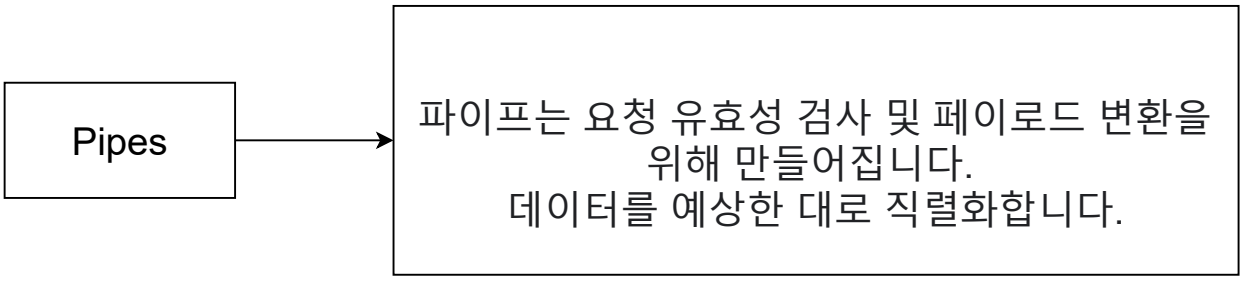
```
 Route {
   path: '/auth/authTest',
   stack: [ [Layer] ],
   methods: { post: true } },
authInfo: undefined,
user:
 User {
   id: 4,
   username: 'john1',
   password:
    '$2a$10$w2yWHwnjuD/j0ig5Eo4f0OM.6U5rf25MAahyDSKkwIWnLiAO9GavS' } }
```

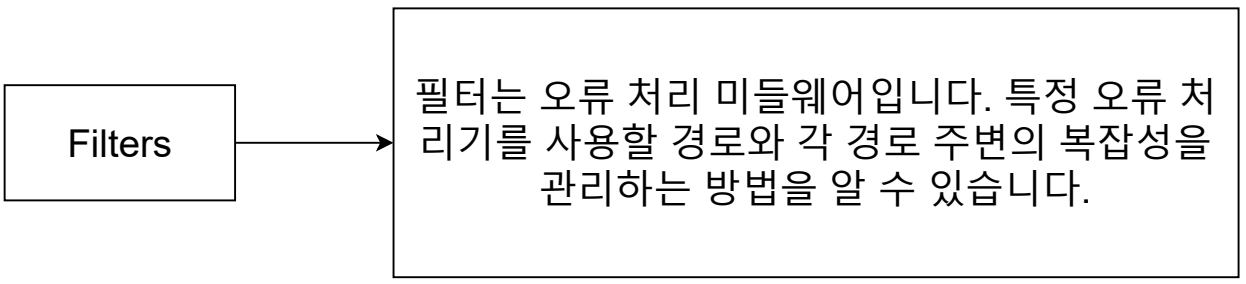
NestJS에서 Middleware들에 대해서

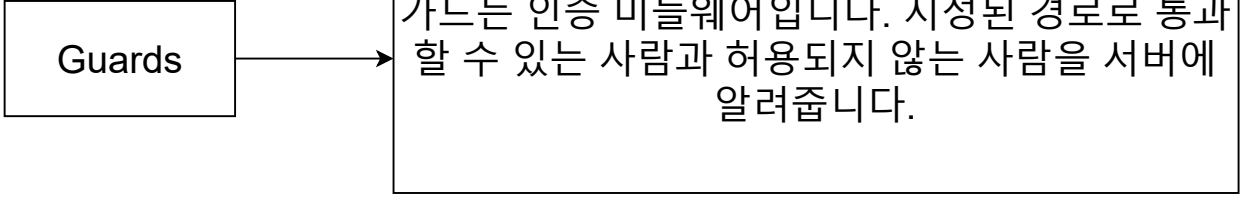
Nest JS에는 여러가지 미들웨어가 있습니다.

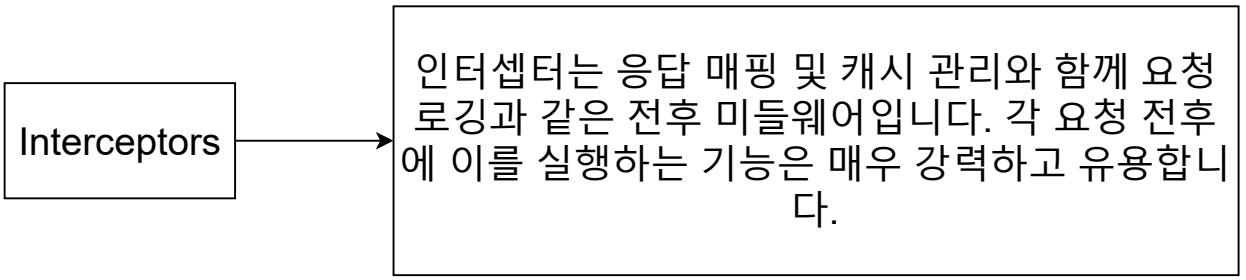
Pipes, Filters, Guards, Interceptors 등의 미들웨어로 취급되는 것들이 있는데 각각 다른 목적을 가지며 사용되고 있습니다.











각각의 미들웨어가 불러지는(called) 순서

middleware -> guard -> interceptor (before) -> pipe -> controller -> service -> controller -> interceptor (after) -> filter (if applicable) -> client

커스텀 데코레이터 생성하기

요청안에 유저가 들어가게 하려면....

```
@Post('/authTest')
@UseGuards(AuthGuard())
authTest(@Req() req) {
    console.log(req);
}
```

```
route:
 Route {
   path: '/auth/authTest',
   stack: [ [Layer] ],
   methods: { post: true } },
authInfo: undefined,
user:
 User {
   id: 4,
   username: 'john1',
   password:
    '$2a$10$w2yWHwnjuD/j0ig5Eo4f0OM.6U5rf25MAahyDSKkwIWnLiAO9GavS' } }
```

req.user 하면 유저 객체를 얻을수 있

req.user가 아닌 바로 user 라는 파라미터로 가져올수있는 방법은 ??

커스텀 데코레이터를 이용하면 됩니다.

```
import { createParamDecorator, ExecutionContext } from "@nestjs/common";
import { User } from "./user.entity";

export const GetUser = createParamDecorator((data, ctx: ExecutionContext): User => {
    const req = ctx.switchToHttp().getRequest();
    return req.user;
});
```

인증된 유저만 게시물 보고 쓸 수 있게 만들기

이번 시간에는 인증된 유저만 게시물을 보고 쓰고 업데이트 하는 작업을 할 수 있게 만들겠습니다.

유저에게 게시물 접근 권한 주기

1. 인증에 관한 모듈을 board 모듈에서 쓸 수 있어야 하기에 board module에서 인증 모듈 imports 해오기 (이렇게 되면 AuthModule에서 export 하는 어떠한 것이든 board Module에서 사용 가능하게 됩니다.)

```
@Module({
  imports: [
    TypeOrmModule.forFeature([BoardRepository]),
    AuthModule
  ],
  controllers: [BoardsController],
  providers: [BoardsService]
})
export class BoardsModule { }
```

2. UseGuards(AuthGuard()) 를 이용해서 이 사람이 요청을 줄 때 올바른 토큰을 가지고 요청을 주는지 본 후에 게시물에 접근 할 권한을 줍니다. 그리고 이 AuthGuard는 각각의 라우트 별로 줄 수도 있고 한번에 하나의 컨트롤러 안에 들어있는 모든 라우트에 줄 수도 있습니다.
현재는 board 컨트롤러 안에 있는 모든 라우트에 AuthGuard를 적용해보겠습니다.

```
@Controller('boards')
@UseGuards(AuthGuard())
export class BoardsController {
  constructor(private boardsService: BoardsService) { }

  @Get()
  getAllTask(): Promise<Board[]> {
```

```
  return this.boardsService.getAllBoards();
}
```

이렇게 되면 이제 올바른 토큰을 넣어서 요청을 주지 않으면 401 Unauthorized 에러가 나게 됩니다.

